16. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_INPUT 200

/* Store background job information */
typedef struct {
    pid_t pid;
    char command[MAX_INPUT];
    int active;
} Job;

Job backgroundJob;

/* Check whether child process has finished */
void monitorJob() {

    if (!backgroundJob.active)
        return;

    int status;

    pid_t result = waitpid(
        backgroundJob.pid,
        &status,
        WNOHANG
    );

    if (result == 0) {

        printf("[Job %d] Still running: %s\n",
               backgroundJob.pid,
               backgroundJob.command);
    }

    else if (result == backgroundJob.pid) {

        backgroundJob.active = 0;

        if (WIFEXITED(status)) {

            printf("[Job %d] Completed with status %d\n",
                   backgroundJob.pid,
                   WEXITSTATUS(status));
        }
        else {
            printf("[Job %d] Terminated abnormally\n",
                   backgroundJob.pid);
```

```
  GNU nano 8.7.1
            printf("[Job %d] Terminated abnormally\n",
                   backgroundJob.pid);
        }
    }
}

/* Launch background command */
void launchBackgroundJob(char *command) {

    pid_t pid = fork();

    if (pid < 0) {

        perror("fork");
        return;
    }

    /* Child process */
    if (pid == 0) {

        /*
         * Remove '&' from command.
         */
        char *ampersand = strchr(command, '&');

        if (ampersand != NULL)
            *ampersand = '\0';

        /*
         * Parse command arguments.
         */
        char *args[20];
        int count = 0;

        char *token = strtok(command, " \t");

        while (token != NULL && count < 19) {

            args[count++] = token;

            token = strtok(NULL, " \t");
        }

        args[count] = NULL;

        /*
         * Execute command.
         */
        execvp(args[0], args);

        perror("Command execution failed");
        exit(1);
```

```
  GNU nano 8.7.1
        exit(1);
    }

    /* Parent process */

    backgroundJob.pid = pid;

    strcpy(backgroundJob.command, command);

    backgroundJob.active = 1;

    /*
     * Do NOT wait here.
     * This allows the prompt to return immediately.
     */
    printf("[Job started] PID: %d\n", pid);
}

int main() {

    char input[MAX_INPUT];

    backgroundJob.active = 0;

    printf("Background Job Manager\n");
    printf("----------------------\n");

    printf("Use '&' to run a command in background.\n");
    printf("Example: sleep 5 &\n");
    printf("Type 'jobs' to monitor the job.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1) {

        /*
         * Check whether background job finished.
         */
        monitorJob();

        printf("shell> ");
        fflush(stdout);

        if (fgets(input,
                  sizeof(input),
                  stdin) == NULL) {

            break;
        }

        input[strcspn(input, "\n")] = '\0';

        /* Ignore empty input */
```

```
  GNU nano 8.7.1
        /* Ignore empty input */
        if (strlen(input) == 0)
            continue;

        /* Exit */
        if (strcmp(input, "exit") == 0) {

            if (backgroundJob.active) {

                printf("Background job is still running.\n");
                printf("Wait for it to complete before exiting.\n");

                continue;
            }

            printf("Exiting shell...\n");
            break;
        }

        /* jobs command */
        if (strcmp(input, "jobs") == 0) {

            if (backgroundJob.active) {

                printf("[Job %d] Running: %s\n",
                       backgroundJob.pid,
                       backgroundJob.command);
            }

            else {

                printf("No active background jobs.\n");
            }

            continue;
        }

        /*
         * Detect background operator '&'
         */
        if (strchr(input, '&') != NULL) {

            if (backgroundJob.active) {

                printf("Only one background job is supported.\n");
                continue;
            }

            launchBackgroundJob(input);

            /*
             * Prompt returns immediately.
```

```
            launchBackgroundJob(input);

            /*
             * Prompt returns immediately.
             */
            continue;
        }

        printf("Foreground command: %s\n", input);
    }

    return 0;
}
```

Result:

```
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$ nano Skill16a.c
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$ gcc Skill16a.c -o s16a
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$ ./s16a
Background Job Manager
----------------------
Use '&' to run a command in background.
Example: sleep 5 &
Type 'jobs' to monitor the job.
Type 'exit' to quit.

shell> sleep 5 &
[Job started] PID: 2314
[Job 2314] Still running: sleep 5 &
shell> jobs
[Job 2314] Running: sleep 5 &
[Job 2314] Completed with status 0
shell> ./filename
Foreground command: ./filename
shell> exit
Exiting shell...
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$
```

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 5
#define MAX_COMMAND 100

/* Job states */
typedef enum {
    RUNNING,
    COMPLETED
} JobState;

/* Job record */
typedef struct {
    int job_id;
    pid_t pid;
    pid_t process_group;
    char command[MAX_COMMAND];
    JobState state;
} Job;

/* Job table */
Job jobTable[MAX_JOBS];
int jobCount = 0;

/* Add a job */
void addJob(pid_t pid, const char *command) {

    if (jobCount >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }

    jobTable[jobCount].job_id = jobCount + 1;
    jobTable[jobCount].pid = pid;
    jobTable[jobCount].process_group = pid;

    strcpy(jobTable[jobCount].command, command);

    jobTable[jobCount].state = RUNNING;

    printf("[Job %d] Added\n",
           jobTable[jobCount].job_id);

    jobCount++;
}

/* Display job table */
void displayJobs() {
```

```c
/* Display job table */
void displayJobs() {

    if (jobCount == 0) {
        printf("Job table is empty.\n");
        return;
    }

    printf("\nJob Table\n");
    printf("-----------------------------------------\n");

    printf("ID\tPID\tPGID\tSTATE\t\tCOMMAND\n");

    printf("-----------------------------------------\n");

    for (int i = 0; i < jobCount; i++) {

        printf("%d\t%d\t%d\t",
               jobTable[i].job_id,
               jobTable[i].pid,
               jobTable[i].process_group);

        if (jobTable[i].state == RUNNING)
            printf("RUNNING\t\t");
        else
            printf("COMPLETED\t");

        printf("%s\n",
               jobTable[i].command);
    }

    printf("-----------------------------------------\n");
}

/* Update job status */
void updateJobStatus() {

    int status;

    for (int i = 0; i < jobCount; i++) {

        if (jobTable[i].state == RUNNING) {

            pid_t result = waitpid(
                jobTable[i].pid,
                &status,
                WNOHANG
            );

            if (result == jobTable[i].pid) {

                jobTable[i].state = COMPLETED;
```

```
  GNU nano 8.7.1
            if (result == jobTable[i].pid) {

                jobTable[i].state = COMPLETED;

                printf("[Job %d] Completed.\n",
                       jobTable[i].job_id);
            }
        }
    }
}

/* Remove completed jobs */
void removeCompletedJobs() {

    int newCount = 0;

    for (int i = 0; i < jobCount; i++) {

        if (jobTable[i].state == RUNNING) {

            jobTable[newCount] = jobTable[i];

            newCount++;
        }
    }

    jobCount = newCount;

    printf("Completed jobs removed.\n");
}

/* Validate job records */
void validateJobs() {

    int valid = 1;

    for (int i = 0; i < jobCount; i++) {

        if (jobTable[i].job_id <= 0 ||
            jobTable[i].pid <= 0 ||
            strlen(jobTable[i].command) == 0) {

            valid = 0;
            break;
        }
    }

    if (valid)
        printf("Job records are valid.\n");
    else
        printf("Invalid job record detected.\n");
```

```
    }

    if (valid)
        printf("Job records are valid.\n");
    else
        printf("Invalid job record detected.\n");
}

/* Launch background job */
void launchJob(const char *command) {

    pid_t pid = fork();

    if (pid < 0) {

        perror("fork");
        return;
    }

    if (pid == 0) {

        /*
         * Create a new process group.
         */
        setpgid(0, 0);

        /*
         * Execute command.
         */
        execlp("sleep", "sleep", "5", NULL);

        perror("exec");
        exit(1);
    }

    /*
     * Parent stores job information.
     */
    addJob(pid, command);
}

int main() {

    printf("Job Table and Process Group Manager\n");
    printf("-----------------------------------\n");

    /* Launch first background job */
    launchJob("sleep 5");

    /* Launch second background job */
    launchJob("sleep 5");
```

```
     */
    addJob(pid, command);
}

int main() {

    printf("Job Table and Process Group Manager\n");
    printf("-----------------------------------\n");

    /* Launch first background job */
    launchJob("sleep 5");

    /* Launch second background job */
    launchJob("sleep 5");

    /* Display jobs */
    displayJobs();

    /* Validate records */
    printf("\nValidating job records...\n");
    validateJobs();

    /* Check status */
    printf("\nChecking job status...\n");
    updateJobStatus();

    displayJobs();

    /*
     * Wait a little so jobs can complete.
     */
    printf("\nWaiting for jobs to complete...\n");
    sleep(6);

    /* Update status again */
    updateJobStatus();

    /* Display completed jobs */
    displayJobs();

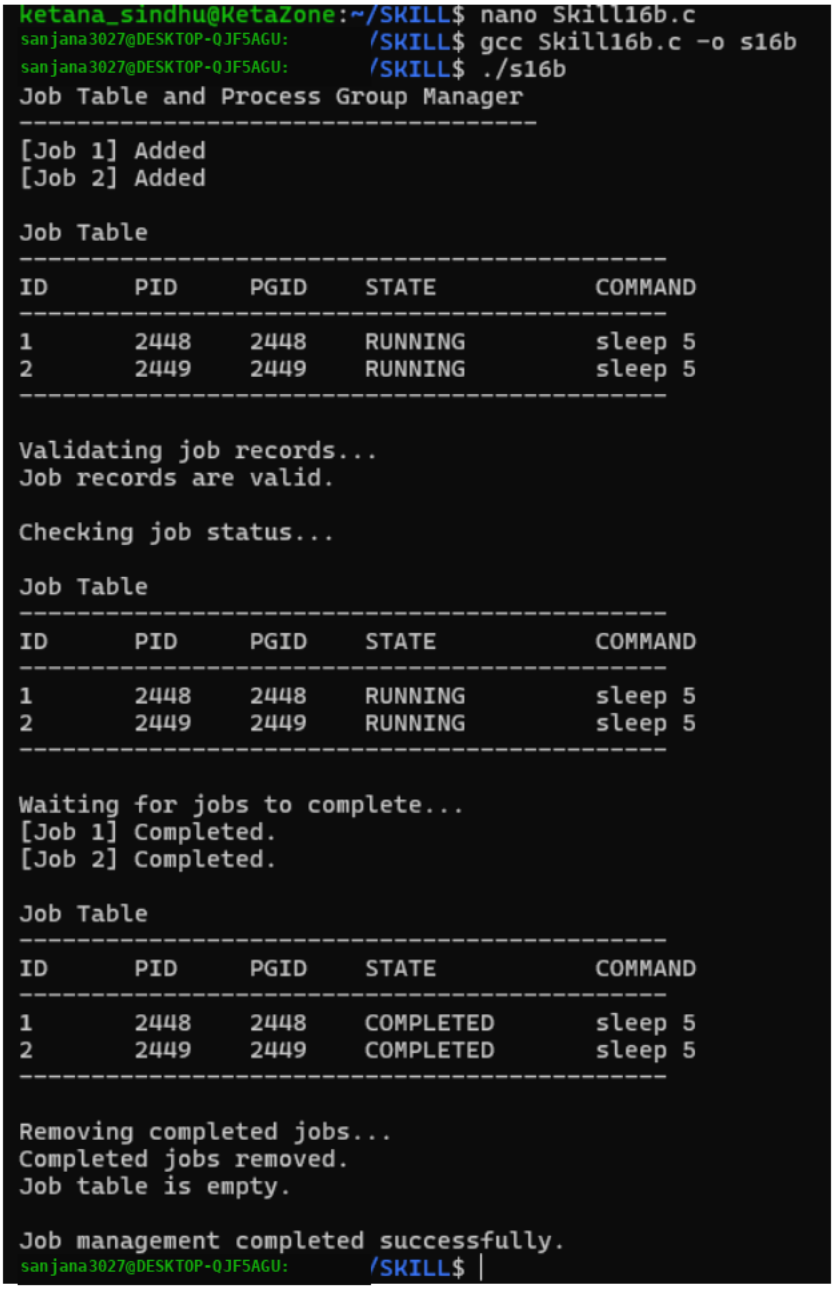
    /* Remove completed jobs */
    printf("\nRemoving completed jobs...\n");
    removeCompletedJobs();

    /* Final table */
    displayJobs();

    printf("\nJob management completed successfully.\n");

    return 0;
}
```

Result:


ketana_sindhu@KetaZone:~/SKILL$ nano Skill16b.c
sanjana3027@DESKTOP-QJF5AGU: /SKILL$ gcc Skill16b.c -o s16b
sanjana3027@DESKTOP-QJF5AGU: /SKILL$ ./s16b
Job Table and Process Group Manager
[Job 1] Added
[Job 2] Added
Job Table
ID PID PGID STATE COMMAND
1 2448 2448 RUNNING sleep 5
2 2449 2449 RUNNING sleep 5
Validating job records...
Job records are valid.
Checking job status...
Job Table
ID PID PGID STATE COMMAND
1 2448 2448 RUNNING sleep 5
2 2449 2449 RUNNING sleep 5
Waiting for jobs to complete...
[Job 1] Completed.
[Job 2] Completed.
Job Table
ID PID PGID STATE COMMAND
1 2448 2448 COMPLETED sleep 5
2 2449 2449 COMPLETED sleep 5
Removing completed jobs...
Completed jobs removed.
Job table is empty.
Job management completed successfully.
sanjana3027@DESKTOP-QJF5AGU: /SKILL$